UNIVERSITY OF CHITTAGONG | Dept. of Computer Science & Engineering

Machine Learning Classifi

With worked numerical examples, diagrams, and intuition-first explanations

Learning Objectives

By the end of this lecture, you will be able to:

- Derive and apply **Bayes' theorem** for probabilistic classification
- Implement the **Naïve Bayes classifier** with the conditional independence assumption
- Understand how **Support Vector Machines** find optimal decision boundaries
- Apply each method on concrete datasets by hand and interpret the results
- Critically compare the three methods and choose the right one for a problem

Contents

Foundations: Probabilistic Reasoning in Classification

Before studying any classifier, we need a common language. All three methods in this lecture reason about *uncertainty* they assign a class label by computing or optimising some notion of probability or confidence over the feature space.

The Classification Problem

Given a dataset $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N}$ where $\mathbf{x}_i \in \mathbb{R}^d$ is a feature vector and $y_i \in \{C_1, C_2, \ldots, C_K\}$ is a class label, we want to learn a function $f : \mathbb{R}^d \to \{C_1, \ldots, C_K\}$ that predicts the correct label for unseen examples.

Definition: Optimal Classifier

The **Bayes-optimal classifier** (the theoretical gold standard) assigns to every point $\mathbf{x}$ the class that maximises the *posterior probability*:

$$f^*(\mathbf{x}) = \arg\max_k \mathbb{P}(C_k \mid \mathbf{x})$$

This minimises the probability of misclassification. In practice, $\mathbb{P}(C_k \mid \mathbf{x})$ is unknown, so different classifiers approximate it in different ways.

Bayesian Classifiers

Bayes' Theorem A Refresher

The cornerstone of probabilistic classification is Bayes' theorem, which *inverts* the direction of conditioning:

Bayes' Theorem

$$\underbrace{\mathbb{P}(C_k \mid \mathbf{x})}_{\text{posterior}} = \frac{\overbrace{\mathbb{P}(\mathbf{x} \mid C_k)}^{\text{likelihood}} \cdot \overbrace{\mathbb{P}(C_k)}^{\text{prior}}}{\underbrace{\mathbb{P}(\mathbf{x})}_{\text{evidence}}}$$

Posterior: probability of class C_k given the observed features $\mathbf{x}$.
Likelihood: how probable are these features if the class is C_k?
Prior: how frequent is class C_k overall?
Evidence: a normalising constant (same for all classes, so often ignored).

Because $\mathbb{P}(\mathbf{x})$ is identical for all classes, classification reduces to:

$$f(\mathbf{x}) = \arg\max_k \mathbb{P}(\mathbf{x} \mid C_k)\,\mathbb{P}(C_k)$$

Bayesian Classifier: Full Joint Distribution

A **Bayesian (generative) classifier** explicitly models $\mathbb{P}(\mathbf{x} \mid C_k)$ as a parametric distribution (commonly a multivariate Gaussian) and learns its parameters from training data.

Quadratic Discriminant Analysis (QDA)

Assume that within each class, features follow a multivariate Gaussian:

$$\mathbb{P}(\mathbf{x} \mid C_k) = \frac{1}{(2\pi)^{d/2}|\Sigma_k|^{1/2}} \exp\left(-\frac{1}{2}(\mathbf{x}-\mu_k)^\top \Sigma_k^{-1}(\mathbf{x}-\mu_k)\right)$$

where μ_k is the class mean and Σ_k is the class covariance matrix. Each class gets its *own* covariance, producing **quadratic** decision boundaries.

Linear Discriminant Analysis (LDA)

If we assume *all classes share the same covariance* $\Sigma_k = \Sigma$, the decision boundaries become **linear**:

$$\delta_k(\mathbf{x}) = \mathbf{x}^\top \Sigma^{-1}\mu_k - \frac{1}{2}\mu_k^\top \Sigma^{-1}\mu_k + \log \mathbb{P}(C_k)$$

Predict class $k^* = \arg\max_k \delta_k(\mathbf{x})$.

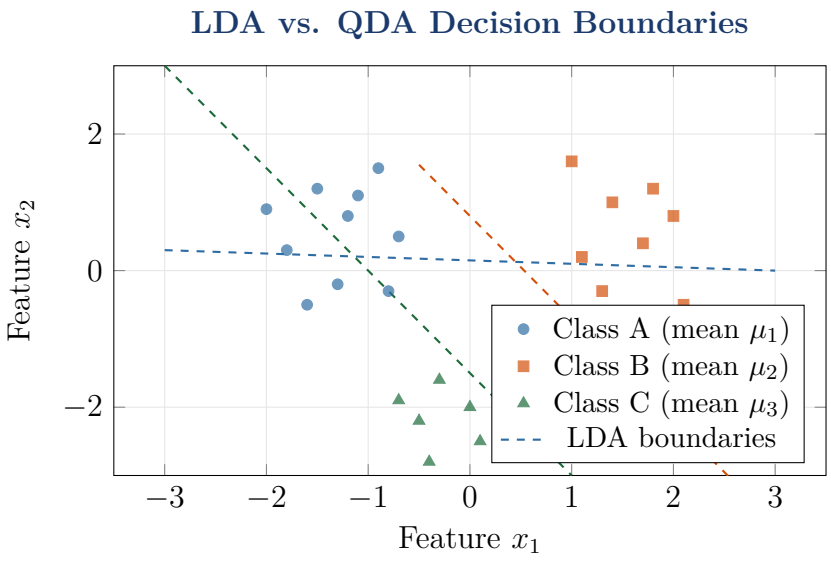

Figure 1: Three-class LDA. Each dashed line is a linear boundary between two classes. The shared covariance assumption forces all boundaries to be linear.

Worked Example Medical Diagnosis

Example 1: Disease Classification

Setting. A patient presents with a blood test result x (glucose level, scaled). Historical data shows:

Class	$\mathbb{P}(C_k)$	Mean μ_k	Std σ_k
Healthy (C_1)	0.70	5.0	1.2
Diabetic (C_2)	0.30	9.5	2.0

New patient: $x = 7.8$ mmol/L. **Which class?**

Step 1 Compute likelihoods (1-D Gaussian):

$$\mathbb{P}(x \mid C_k) = \frac{1}{\sigma_k\sqrt{2\pi}} \exp\left(-\frac{(x-\mu_k)^2}{2\sigma_k^2}\right)$$

$$\mathbb{P}(x{=}7.8 \mid C_1) = \frac{1}{1.2\sqrt{2\pi}} e^{-(7.8-5.0)^2/(2\times 1.44)} = \frac{1}{3.005} e^{-2.722} = 0.0177$$

$$\mathbb{P}(x{=}7.8 \mid C_2) = \frac{1}{2.0\sqrt{2\pi}} e^{-(7.8-9.5)^2/(2\times 4.0)} = \frac{1}{5.013} e^{-0.361} = 0.1397$$

Step 2 Apply Bayes' theorem (ignore denominator):

$$\text{Score}(C_1) = 0.0177 \times 0.70 = 0.01239$$

$$\text{Score}(C_2) = 0.1397 \times 0.30 = 0.04191$$

Step 3 Predict: $0.04191 > 0.01239$, so predict $\boxed{C_2 : \text{Diabetic}}$.

Posterior probabilities (after normalising):

$$\mathbb{P}(C_1 \mid x) = \frac{0.01239}{0.05430} = 22.8\%, \qquad \mathbb{P}(C_2 \mid x) = \frac{0.04191}{0.05430} = 77.2\%$$

Naïve Bayesian Classifiers

The Curse of Dimensionality Motivation

A full Bayesian classifier estimates $\mathbb{P}(\mathbf{x} \mid C_k)$ jointly over *all* d features. If each feature takes v values, storing the full joint table requires v^d entries per class exponential in d. With $d = 50$ binary features, that is $2^{50} \approx 10^{15}$ parameters.

The Naïve Bayes Assumption

Assume features are **conditionally independent** given the class:

$$\mathbb{P}(\mathbf{x} \mid C_k) = \mathbb{P}(x_1, x_2, \ldots, x_d \mid C_k) \approx \prod_{j=1}^{d} \mathbb{P}(x_j \mid C_k)$$

This reduces the parameter count from v^d to $v \times d$ per class a massive saving.

The classification rule becomes:

$$\boxed{f(\mathbf{x}) = \arg\max_k \mathbb{P}(C_k) \prod_{j=1}^{d} \mathbb{P}(x_j \mid C_k)}$$

In practice we use **log-probabilities** to avoid numerical underflow:

$$f(\mathbf{x}) = \arg\max_k \left[\log \mathbb{P}(C_k) + \sum_{j=1}^{d} \log \mathbb{P}(x_j \mid C_k) \right]$$

Parameter Estimation

For **categorical features** (text, discrete attributes):

$$\hat{\mathbb{P}}(x_j = v \mid C_k) = \frac{\text{count}(x_j = v, C_k) + \alpha}{\text{count}(C_k) + \alpha |\mathcal{V}_j|}$$

where α is the Laplace smoothing parameter (typically $\alpha = 1$) to avoid zero probabilities for unseen feature values.

For **continuous features** (Gaussian Naïve Bayes):

$$\mathbb{P}(x_j \mid C_k) = \mathcal{N}(x_j;\, \mu_{jk},\, \sigma_{jk}^2)$$

where μ_{jk} and σ_{jk} are the mean and standard deviation of feature j within class k.

Worked Example Spam E-mail Classification

Example 2: Naïve Bayes for Spam Detection

Training set: 10 emails (6 Ham, 4 Spam). Count of keyword occurrences:

Word	Count in Ham	Count in Spam	Vocab size = 5
`free`	1	4	
`money`	0	3	
`meeting`	5	0	
`lunch`	4	1	
`offer`	0	2	
Total words	**10**	**10**	

Priors: $\mathbb{P}(\text{Ham}) = 6/10 = 0.6$; $\mathbb{P}(\text{Spam}) = 4/10 = 0.4$

With Laplace smoothing ($\alpha = 1$, $|\mathcal{V}| = 5$):

	$\mathbb{P}(w \mid \text{Ham})$	$\mathbb{P}(w \mid \text{Spam})$
`free`	$(1+1)/(10+5) = 2/15$	$(4+1)/(10+5) = 5/15$
`money`	$(0+1)/(10+5) = 1/15$	$(3+1)/(10+5) = 4/15$
`meeting`	$(5+1)/(10+5) = 6/15$	$(0+1)/(10+5) = 1/15$
`lunch`	$(4+1)/(10+5) = 5/15$	$(1+1)/(10+5) = 2/15$
`offer`	$(0+1)/(10+5) = 1/15$	$(2+1)/(10+5) = 3/15$

New email: "*free money offer*" classify it.

Compute log-scores:

$$\begin{aligned}\log \text{Score(Ham)} &= \log(0.6) + \log(2/15) + \log(1/15) + \log(1/15) \\ &= -0.511 + (-2.015) + (-2.708) + (-2.708) = -7.942 \\ \log \text{Score(Spam)} &= \log(0.4) + \log(5/15) + \log(4/15) + \log(3/15) \\ &= -0.916 + (-1.099) + (-1.322) + (-1.609) = -4.946\end{aligned}$$

$-4.946 > -7.942$, so predict $\boxed{\text{Spam}}$.

Variants of Naïve Bayes

Variant	Feature type	Typical use
Bernoulli NB	Binary (word present/absent)	Short text, document classification
Multinomial NB	Integer counts	Bag-of-words, frequency-based text
Gaussian NB	Real-valued	Sensor data, measurements
Complement NB	Count-based	Imbalanced text datasets

Strengths and Weaknesses

Key Takeaways Naïve Bayes

Strengths

- Extremely fast to train and predict
- Works well with small datasets
- Robust to irrelevant features
- Handles missing values naturally
- Excellent baseline for text tasks

Weaknesses

- Independence assumption is rarely true
- Poor probability calibration
- Feature correlations are ignored
- "Zero frequency" problem (fixed by smoothing)

Support Vector Machines (SVM)

Intuition: Finding the Best Separator

Naïve Bayes asks "*what is the probability of each class?*" An SVM asks a fundamentally different question: "*which hyperplane separates the classes with the maximum gap?*"

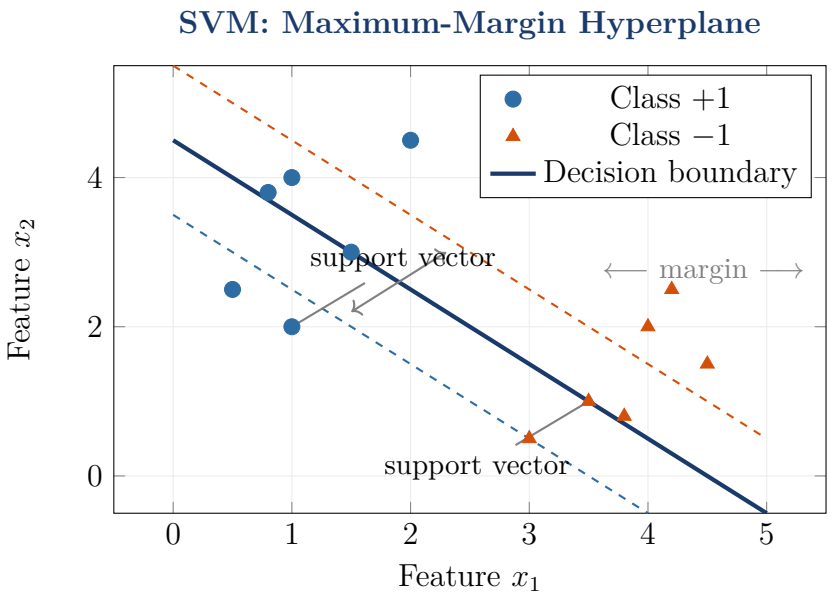

Figure 2: SVM finds the hyperplane (solid line) that maximises the margin the gap between the dashed lines. Only the *support vectors* (points on or inside the margin) determine the boundary.

The Hard-Margin SVM (Linearly Separable Case)

Let training data be $\{(\mathbf{x}_i, y_i)\}$ where $y_i \in \{-1, +1\}$. A hyperplane is defined by $\mathbf{w}^\top \mathbf{x} + b = 0$.

The **geometric margin** of the hyperplane is $\frac{2}{\|\mathbf{w}\|}$. Maximising the margin is equivalent to minimising $\|\mathbf{w}\|$, subject to correct classification:

Hard-Margin SVM Primal Optimisation Problem

$$\min_{\mathbf{w}, b} \quad \frac{1}{2}\|\mathbf{w}\|^2$$

$$\text{subject to:} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 \quad \forall i = 1, \ldots, N$$

This is a **convex quadratic programme** with a unique global optimum.

Soft-Margin SVM (Non-Separable Case)

Real data is rarely linearly separable. We introduce **slack variables** $\xi_i \geq 0$ that allow points to violate the margin:

Soft-Margin SVM Primal Problem

$$\min_{\mathbf{w}, b, \xi} \quad \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{N} \xi_i$$

$$\text{subject to:} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0 \quad \forall i$$

$C > 0$ is the regularisation parameter:

- Large C: penalise misclassifications heavily (low bias, high variance)
- Small C: tolerate errors to get a wider margin (high bias, low variance)

The Dual Problem and Support Vectors

Using Lagrange multipliers $\alpha_i \geq 0$, the dual problem is:

$$\max_{\alpha} \quad \sum_{i=1}^{N} \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j \mathbf{x}_i^\top \mathbf{x}_j$$

$$\text{s.t.} \quad \sum_{i=1}^{N} \alpha_i y_i = 0, \quad 0 \leq \alpha_i \leq C$$

The solution gives $\mathbf{w} = \sum_i \alpha_i y_i \mathbf{x}_i$. Points with $\alpha_i > 0$ are the **support vectors** the only training points that matter for the decision boundary.

The Kernel Trick Non-linear SVMs

When data is not linearly separable in input space, we implicitly map features to a higher-dimensional space via a **kernel function** $K(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^\top \phi(\mathbf{x}_j)$, and replace all inner products $\mathbf{x}_i^\top \mathbf{x}_j$ with $K(\mathbf{x}_i, \mathbf{x}_j)$:

Kernel	Formula	Use case
Linear	$K(\mathbf{x}, \mathbf{x}') = \mathbf{x}^\top \mathbf{x}'$	Linearly separable, high-dim text
Polynomial	$K(\mathbf{x}, \mathbf{x}') = (\gamma \mathbf{x}^\top \mathbf{x}' + r)^d$	Moderate non-linearity
RBF / Gaussian	$K(\mathbf{x}, \mathbf{x}') = \exp(-\gamma \lVert \mathbf{x} - \mathbf{x}' \rVert^2)$	Most common; good default
Sigmoid	$K(\mathbf{x}, \mathbf{x}') = \tanh(\gamma \mathbf{x}^\top \mathbf{x}' + r)$	Neural-network-like

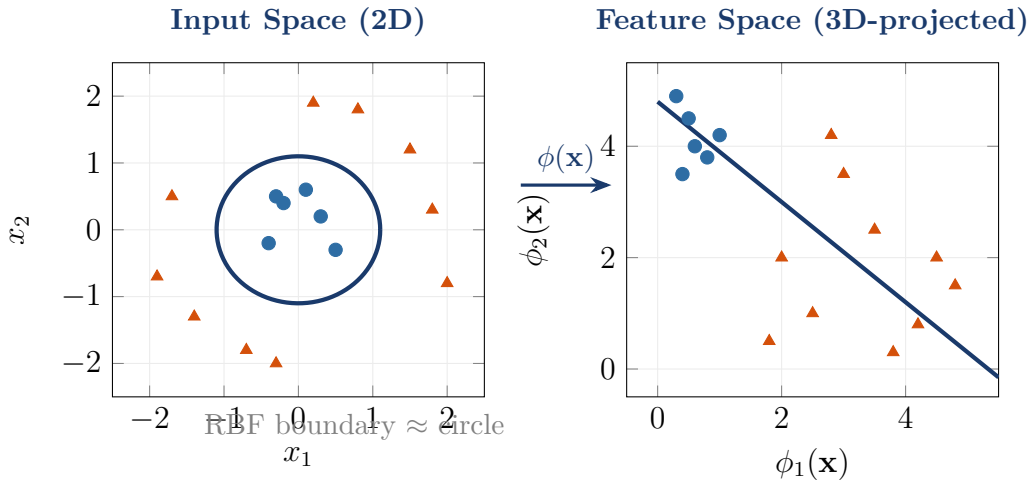

Figure 3: The kernel trick. Left: data is not linearly separable in 2D. Right: after mapping $\phi(\mathbf{x})$ to a higher-dimensional space (the RBF kernel implicitly uses infinite dimensions!), the classes *are* linearly separable.

Worked Example SVM by Hand (2D)

Example 3: Finding the Maximum-Margin Hyperplane

Dataset (4 points, 2D):

Point	x_1	x_2	Label y
$\mathbf{x}_1$	1	1	$+1$
$\mathbf{x}_2$	2	2	$+1$
$\mathbf{x}_3$	3	1	-1
$\mathbf{x}_4$	4	2	-1

Step 1 Identify candidate support vectors.
By inspection, $\mathbf{x}_2 = (2, 2)$ and $\mathbf{x}_3 = (3, 1)$ are the closest points from each class.
Step 2 Set up the margin constraints.
For support vectors: $y_i(\mathbf{w}^\top \mathbf{x}_i + b) = 1$.

$$+1 \cdot (2w_1 + 2w_2 + b) = 1 \quad \Rightarrow \quad 2w_1 + 2w_2 + b = 1$$

$$-1 \cdot (3w_1 + 1w_2 + b) = 1 \quad \Rightarrow \quad 3w_1 + w_2 + b = -1$$

Step 3 Solve the system.
Subtracting: $(3w_1 + w_2 + b) - (2w_1 + 2w_2 + b) = -1 - 1$

$$w_1 - w_2 = -2 \quad \Rightarrow \quad w_1 = w_2 - 2$$

For a symmetric perpendicular bisector, choose $w_1 = 1,\ w_2 = 3$, then:

$$2(1) + 2(3) + b = 1 \ \Rightarrow \ b = 1 - 8 = -7$$

Decision boundary: $x_1 + 3x_2 - 7 = 0$

Step 4 Compute the margin:

$$\text{margin} = \frac{2}{\|\mathbf{w}\|} = \frac{2}{\sqrt{1^2 + 3^2}} = \frac{2}{\sqrt{10}} \approx 0.632$$

Step 5 Predict a new point $\mathbf{x}_{\text{new}} = (2.5, 1.5)$:

$$\text{score} = 1 \cdot 2.5 + 3 \cdot 1.5 - 7 = 2.5 + 4.5 - 7 = 0.0 \ \Rightarrow \ \text{On the boundary}$$

Try $\mathbf{x}_{\text{new}} = (1.5, 2)$: score $= 1.5 + 6 - 7 = 0.5 > 0 \ \Rightarrow \ \boxed{+1}$

Multi-Class SVM

Standard SVM is binary. For K classes, two strategies are used:

Multi-Class SVM Strategies

One-vs-Rest (OvR):
Train K SVMs, each separating class k from all others. Predict the class whose SVM gives the highest positive score. *(Most common)*

One-vs-One (OvO):
Train $\binom{K}{2}$ SVMs, one per pair of classes. Take the majority vote among all classifiers.

Comparison & Practical Guide

Criterion	Bayesian Classifier	Naïve Bayes	SVM
Model type	Generative	Generative	Discriminative
Key assumption	Gaussian features	Feature independence	Max-margin boundary
Training speed	Medium	**Very fast**	Slow ($O(N^3)$)
Prediction speed	Fast	**Very fast**	Fast
Small data	Good	Good	**Excellent**
Large data	Good	**Excellent**	Poor (memory)
High-dim features	Poor	**Excellent**	**Excellent**
Non-linear data	Limited (QDA)	Poor	**Excellent (kernels)**
Probability output	**Yes (calibrated)**	Yes (poorly calibrated)	Requires Platt scaling
Interpretability	**High**	**High**	Low (kernel SVMs)
Noise robustness	Medium	**Sensitive**	**High (soft margin)**

Decision Flowchart: Which Method to Choose?

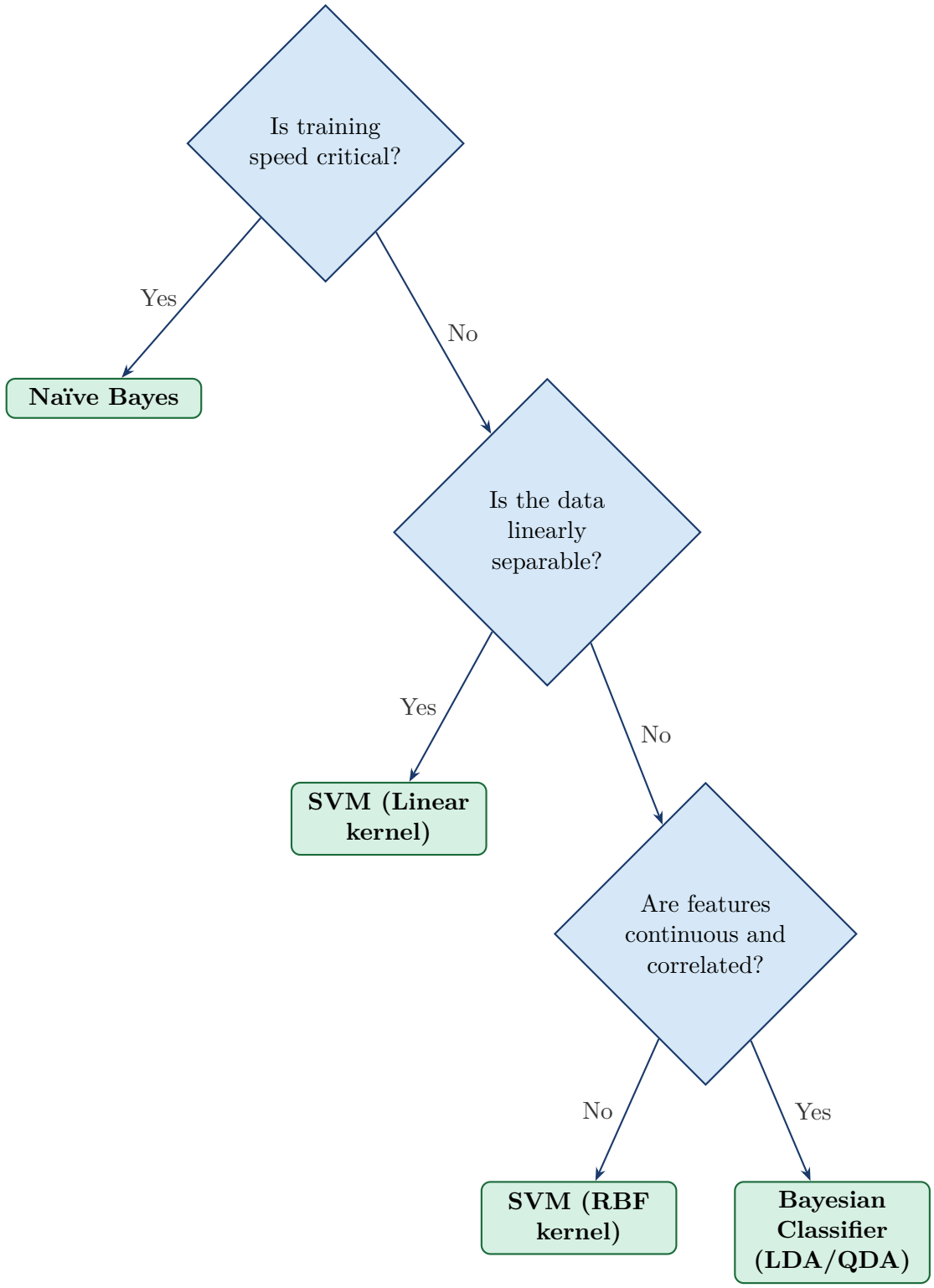

Figure 4: A simplified guide for choosing between the three classifiers. In practice, always cross-validate multiple methods.

Comprehensive Worked Example All Three Methods

Example 4: Iris-Style Classification (Simplified)

Dataset: 2 features (petal length x_1, petal width x_2), 2 classes.

ID	x_1	x_2	Class	Split
1	1.5	0.3	Setosa (+1)	Train
2	1.3	0.2	Setosa (+1)	Train
3	4.7	1.4	Virginica (−1)	Train
4	4.5	1.5	Virginica (−1)	Train
5	1.4	0.2	Setosa (+1)	Train
6	4.6	1.3	Virginica (−1)	Train
?	2.0	0.5	**???**	**Test**

Test point: $\mathbf{x}_{\text{test}} = (2.0,\ 0.5)$

Method A Bayesian Classifier (Gaussian LDA):
Class means: $\mu_+ = (1.4, 0.23)$; $\mu_- = (4.6, 1.4)$.
Shared (pooled) variance (simplified, diagonal): $\sigma_1^2 \approx 0.01$, $\sigma_2^2 \approx 0.02$.
Priors: $\mathbb{P}(+1) = \mathbb{P}(-1) = 0.5$.
Mahalanobis distance to each class:

$$d(+1) = \frac{(2.0 - 1.4)^2}{0.01} + \frac{(0.5 - 0.23)^2}{0.02} = 36 + 3.6 = 39.6$$

$$d(-1) = \frac{(2.0 - 4.6)^2}{0.01} + \frac{(0.5 - 1.4)^2}{0.02} = 676 + 40.5 = 716.5$$

$d(+1) < d(-1) \Rightarrow$ Predict $\boxed{+1 : \text{Setosa}}$.

Method B Naïve Bayes (Gaussian):
$\mathbb{P}(x_1 = 2.0 \mid +1) = \mathcal{N}(2.0;\ 1.4,\ 0.1) \approx 0.00248$; $\mathbb{P}(x_1 = 2.0 \mid -1) \approx 0.0$
Since $\mathbb{P}(x_1 = 2.0 \mid -1) \approx 0$, log-score for -1 is $-\infty$.
Predict $\boxed{+1 : \text{Setosa}}$.

Method C Linear SVM (conceptual):
The boundary is approximately $x_1 = 3.05$ (midpoint between classes).
$x_{\text{test},1} = 2.0 < 3.05 \Rightarrow$ Predict $\boxed{+1 : \text{Setosa}}$.

All three methods agree. For this well-separated dataset, the choice of method has no impact on this prediction.

Summary

the class-conditional distributions (Gaussian). LDA assumes shared covariance (linear boundaries); QDA allows per-class covariances (quadratic boundaries). Requires enough data per class to estimate parameters reliably.

2. **Naïve Bayes** is a fast, scalable special case that assumes conditional independence of features given the class. Despite this strong (often violated) assumption, it performs surprisingly well on text classification, spam filtering, and high-dimensional discrete data.

3. **SVMs** take a completely different approach they find the *maximum-margin* hyperplane. The soft-margin version handles noise via the penalty parameter C. The *kernel trick* allows SVMs to create non-linear boundaries without ever explicitly computing the high-dimensional feature map.

4. **Choosing a method:** No single classifier dominates all tasks. Naïve Bayes is your go-to baseline when speed matters or data is sparse and discrete. SVMs excel on small-to-medium datasets with complex boundaries. Bayesian classifiers shine when probabilistic outputs are essential.

Further Reading

- Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer. Chapters 4, 7.
- Mitchell, T. M. (1997). *Machine Learning*. McGraw-Hill. Chapter 6 (Bayesian learning).
- Cortes, C. & Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20(3), 273297.
- Ng, A. Y. & Jordan, M. I. (2001). On discriminative vs. generative classifiers. *NeurIPS*.
- Scikit-learn documentation: https://scikit-learn.org/stable/supervised_learning.html

End of Lecture Notes

University of Chittagong • Department of CSE
CSE817: Data Engineering • ML Classification Methods

Questions? Post on Google Classroom or visit office hours.